Castas Android

Auditoria retrospectiva

Da referência web ao candidato de release

O que ocorreu, como as evidências mudaram e quais controles levar para o próximo projeto.

40
perguntas por idioma

26-36
APIs testadas nos extremos

21
skills na coleção atual

Navegue pelo relatório

02-03	Parecer, escopo e método
04-08	Cronologia, arquitetura, paridade e correções
09-14	Emulador, testes, CI, documentação e privacidade
15-17	Teste pessoal, tablets e publicação
18-21	Skills, SDD, OpenSpec e modelos
22-27	Fábrica de software, plano de ação e evidência visual

09 de novembro de 2026. Android **9ce150a**, referência web **791ef01** e coleção Teknolista **95abba6**. Revisões históricas são identificadas ao longo do relatório.

PARECER

Um candidato local verificável

A versão Android avançou da primeira implementação para um candidato testado em dois emuladores. O pipeline do GitHub passou no commit atual. A validação do canal de distribuição ainda é uma etapa própria.

Conclusão	Evidência e próximo passo
Produto	Sete jornadas nativas, três idiomas e o mesmo cálculo da web. O fechamento corrigiu rotação, armazenamento, progresso, contraste e compatibilidade com a API mínima.
Verificação	A auditoria de fechamento registrou 9 testes JVM e 5 instrumentados em cada um de dois AVDs. Nesta rodada, confirmei a paridade do domínio, os artefatos locais, a validação OpenSpec e o CI remoto.
Release	O AAB local está sem assinatura de produção. Confirme direitos dos ativos, assinatura, política pública e instalação/atualização em aparelho pelo canal escolhido.
Processo	A maior alavanca é rodar uma fatia real cedo. Fixe a origem da regra, confirme o ambiente que executa o teste e vincule a prova ao snapshot.

Três controles com retorno imediato

01. Faça preflight no mesmo contexto que vai executar o teste, incluindo permissões, SDK, dispositivo e capacidade de boot.

02. Faça uma instalação limpa das skills da revisão escolhida e exija seleção explícita e evidência de aplicação nas etapas que dependem delas.

03. Ao adaptar um projeto de referência, valide as afirmações na plataforma de destino. O README atual do Android menciona localStorage e aponta para um arquivo de licença inexistente naquele caminho.

> **Conclusão de qualidade**
>
> A prontidão técnica local está sustentada para as fronteiras exercitadas. A decisão de publicar depende de comprovar o pacote que o usuário receberá e resolver as pendências de release.

ESCOPO E MÉTODO

O que a auditoria comparou

A pergunta é como melhorar o processo de desenvolvimento. Por isso a unidade de análise é a cadeia de decisão e evidência, do requisito até o efeito observado.

Fonte	Como entrou no relatório
Histórico do Codex	Conversa Castas Android 01a1b8ae…52e86. Usei os pedidos originais, as revisões, os resultados registrados e os metadados de modelo para reconstruir a sequência.
Repositórios	Android 2a133da, 14ab318 e 9ce150a. Web ca2ca90, 92d4b8d e 791ef01. Coleção histórica 547b47b e atual 95abba6.
Arquivos do projeto	Duas auditorias, cinco pilares, contrato de UI, specs, changes arquivadas, testes, manifesto, workflow e README.
Artefatos e execução	Hashes do APK/AAB, leitura do pacote, teste comparativo TypeScript/Kotlin, validação OpenSpec, checagem KVM e API pública do GitHub Actions.
Documentação de referência	.etc do Castas Web, guia SDD atualizado e documentação oficial aplicável às ferramentas e aos gates de publicação.

Como ler as evidências

Observado agora indica uma checagem desta rodada. **Histórico verificado** indica um registro anterior compatível com o código e a revisão indicada. **Pendente** indica uma fronteira que ainda exige execução ou decisão.

A nova auditoria examinou o produto e o processo em leitura. Os ensaios novos rodaram em arquivos temporários ou inspecionaram artefatos existentes. A execução Gradle e as suítes Android em emulador pertencem aos registros anteriores e ao CI consultado.

> **Regra de interpretação**
>
> Se duas fontes divergem, identifique o dono do contrato e a revisão antes de decidir. Uma tela capturada, um teste e uma spec descrevem evidências diferentes.

CRONOLOGIA / PRIMEIRA ENTREGA

Da intenção ao primeiro APK

O pedido autorizava implementar o Castas 100% nativo, com o comportamento da web e a experiência de navegação do Android.

Marco	Decisão, fato observado e efeito
07/11 · início	Leitura do código React, wireframes, questionário, catálogos, assets e lógica. O scorer e o documento Likert diziam 3/2/1/0. O wireframe Sobre dizia 4/3/2/1.
Bootstrap	O ambiente não tinha SDK configurado para a execução inicial e o Gradle falhou ao carregar libnative-platform.so. O OpenSpec 1.16.0 criou a estrutura e a change, mas a integração Codex encontrou EROFS em .agents/skills.
Contrato inicial	A change castas-native-android reuniu proposta, design, delta e tarefas. Os pilares e o contrato de UI serviram como contexto e referência.
2a133da	Entrega Kotlin/Compose em develop. O build produziu APK, teste instrumentado compilado e AAB local. A primeira tentativa de dispositivo ficou offline; a change ainda estava em 6/12 tarefas.
08/11 · primeira auditoria	O anfitrião confirmou KVM. Um AVD API 35 rodou a jornada e reproduziu a perda da pergunta 2 após girar. Lint detectou NewApi e a inspeção visual encontrou contraste quase ilegível.

O que funcionou

Houve transferência de conteúdo e regra de pontuação, uma implementação realmente nativa e revisão em dois eixos. O código e os artefatos existiam para que a auditoria pudesse testar as promessas, localizar divergências e definir a próxima change.

> **O erro de sequência**
>
> A validação de ambiente e uma jornada mínima no dispositivo deveriam ter acontecido antes da maior parte da implementação. A ausência desse checkpoint permitiu que incompatibilidade e lifecycle chegassem ao fim da primeira entrega.

CRONOLOGIA / FECHAMENTO

Do reparo ao estado atual

Depois da primeira auditoria, o usuário autorizou as correções e a preparação local de release, mantendo Android 8.0 como mínimo.

Marco	O que mudou
08/11 · 14ab318	ViewModel e persistência corrigidos, seletor de emojis ampliado, UI adaptada, API alvo 36 e duas changes arquivadas. Builds, lint, JVM, emuladores e AAB verificados localmente.
Web · 92d4b8d	O wireframe Sobre foi ajustado para 3/2/1/0. O scorer já usava essa escala. O commit corrigiu a descrição e não precisou transformar resultados.
GitHub Flow	develop foi integrado por fast-forward em main. Os primeiros runs de 14ab318 falharam. O run 42520061733 parou em sdkmanager antes do Gradle.
4a26b21 → 236fa5e	O PR #1 instalou o SDK explicitamente e alinhou os actions do runner. Os checks do PR e o run pós-merge de main concluíram com sucesso. A branch temporária foi removida após a integração.
c1b6adb / bce42c0	Ajuste de .gitignore e atualização posterior do README. A reutilização de texto da web reintroduziu uma afirmação de localStorage no README Android.
09/11 · 9ce150a	954002e retirou .agents do Git e 9ce150a retirou o lockfile. O HEAD atual continua com os artefatos OpenSpec versionados e tem CI verde.

Conferi que o código do app, o wrapper, os arquivos Gradle e o preflight do fechamento coincidem com o HEAD atual. As mudanças posteriores alteraram CI, documentação e a distribuição das instruções do agente.

> **Lição de manutenção**
>
> A evidência de fechamento continua útil quando seu escopo é rastreável. Reavalie o que mudou depois: documentação, bootstrap de clone e configuração de runner podem regredir mesmo quando o binário continua igual.

ARQUITETURA

Uma experiência Android nativa

O projeto adotou um módulo de app com Kotlin, Compose/Material 3, Navigation Compose e Preferences DataStore. O domínio de pontuação e formatação é Kotlin puro.

Camada	Responsabilidade e limite
Catálogos locais	Perguntas e textos em PT-BR, EN e ES, mais imagens e um catálogo de emojis. Os IDs e as dimensões das perguntas derivam da ordem dos arrays.
Compose e navegação	Home, teste, resultado, Sobre, fanatismo político, termos e privacidade. O app usa navegação e diálogo nativos.
ViewModel	Mantém a sessão incompleta em memória enquanto o processo vive. Controla transição, seleção única e persistência do resultado.
Domínio	Cinco dimensões de oito respostas. Soma até 24 por dimensão, normaliza e arredonda. A grade usa cinco células por dimensão.
DataStore	Grava data e cinco escores do último resultado completo em uma operação atômica. Armazena a preferência de idioma.
Saída explícita	Copiar envia a representação localizada para o clipboard. Links externos abrem por ação do usuário.

Paridade com espaço para a plataforma

O contrato comum inclui ordem das perguntas, regra de cálculo, idiomas e ações. O Android usa Voltar do sistema, controles nativos, renderização local de emojis e armazenamento próprio. Registre essas adaptações como decisões, com exemplos de como testá-las.

> **Fronteira importante**
>
> Rotação com o processo vivo preserva a sessão. Sair do teste ou recriar o processo descarta respostas incompletas conforme a spec. O resultado completo só abre após a gravação. São três cenários com expectativas distintas.

REFERÊNCIA E PARIDADE

Do wireframe ao teste cruzado

A pasta .etc foi consultada e forneceu a especificação 5PRO, wireframes, mockup, assets, questionário, escala, estrutura do resultado e texto de cópia.

Observado agora	Como verifiquei
120 traduções	Executei a importação do catálogo TypeScript e comparei 40 entradas por idioma com os arrays Android. Confirmei ordem e equivalência do agrupamento calculado pelo índice.
25 totais por dimensão	Executei o scorer web e compilei o domínio Kotlin em /tmp. Cobri as somas possíveis de 0 a 24 e comparei ambas as saídas com um oráculo aritmético.
101 limiares de emoji	Comparei a contagem de símbolos positivos para todos os escores de 0 a 100. Comparei também a formatação completa de um vetor sintético de cópia.
3 imagens	Confirmei identidade byte a byte do ícone e das duas imagens com os arquivos de referência.

O que a documentação web ainda não dizia

A aplicação React esclareceu o contador por dimensão, o bloqueio de entrada durante a transição e a duração do feedback de cópia. A portabilidade exigiu novos contratos para rotação, abandono, morte de processo, falha de armazenamento, fonte de emojis, acessibilidade e política de distribuição.

> **Padrão reutilizável de portabilidade**
>
> Para cada regra importante, registre fonte e revisão, divergência encontrada, decisão, cenário e evidência. Para a escala Likert, a mudança corrigiu o texto de referência e o teste protegeu a regra executável. Para novas plataformas, reuse os mesmos vetores e teste a experiência no alvo.

ACHADOS DA PRIMEIRA AUDITORIA

Como os defeitos foram fechados

Problema comprovado	Correção e prova correspondente
Rotação · alta	A pergunta voltava à primeira porque o estado morava na composição. O ViewModel mantém o progresso na recriação. Um check de orientação preservou a pergunta 2.
Persistência · alta	A UI exibia sucesso antes de gravar. O fluxo agora aguarda DataStore e oferece retry. Teste unitário exerce falha e preservação do anterior; teste de integração reabre a store.
API mínima · bloqueava lint	O tema base usava um atributo de API 27 com minSdk 26. A remoção resolveu NewApi. A jornada rodou em API 26 no fechamento.
Contraste e adaptação	O botão Alterar Emoji tinha contraste aproximado de 1,15:1 na primeira captura. O conteúdo passou a ter cor explícita, e a grade usa cinco células escaláveis.
Paridade de interação	Contagem dentro da dimensão, toque duplo, estado durante transição, catálogo nativo de emoji e confirmação da cópia foram ajustados. Cenários protegem transição e reabertura.
Prontidão de release	Alvo 36, wrapper com checksum, matriz JDK/AGP/SDK, manifesto e exclusões de backup. O release foi exercitado por splits com assinatura descartável.

Antes desses achados, também houve correções de nomes de recursos, alvos JVM, tipos do cache e imports de teste. Esses erros mostram o custo de validar tarde a toolchain e a primeira fatia.

> **Lição de testes**
>
> O primeiro teste Compose passava mesmo com o contador errado, pois codificava a implementação em vez do contrato. Derive a asserção de uma regra aceita. No Castas, a pergunta 9 deve mostrar a primeira questão da dimensão seguinte e progresso total 9.

INCIDENTE DO EMULADOR

Onde o diagnóstico se desviou

O relato inicial atribuiu a ausência de /dev/kvm ao computador. A comparação entre os ambientes mostrou uma fronteira de execução.

Camada	O que ocorreu e o que se aprende
Sandbox	O device node /dev/kvm ficou invisível e parte da configuração estava em caminhos restritos. Uma observação no sandbox descreve aquele contexto.
Anfitrião	O host expôs /dev/kvm e o comando emulator -accel-check confirmou KVM utilizável. A mesma distinção foi revalidada nesta auditoria.
Boot e ADB	O emulador foi iniciado com -accel off em uma tentativa, inclusive no host. O ADB permaneceu offline no período observado. A espera também usava status, variável especial do Zsh, e não forneceu uma medição confiável.
Espaço e imagem	O AVD antigo teve limitação de espaço no volume de dados. No fechamento, a partição gravável foi colocada em outro volume e as imagens API 26 e API 36 foram tratadas separadamente.
ANR e captura	O Android convidado apresentou ANR. No API 36, o teste de comportamento foi separado da captura opcional e passou numa imagem de 16 KiB com a janela virtual definida.

O significado técnico de cada sinal

KVM acelera CPU virtual no Linux. SwiftShader é renderização de GPU por software e pode coexistir com KVM. AVD é a configuração da máquina virtual. ADB é o canal para instalar e falar com o Android convidado.

> **Causa com limites**
>
> A ausência de KVM no sandbox foi comprovada. A imagem, o espaço e o modo de aceleração também afetaram as tentativas. O histórico sustenta que separar essas variáveis melhorou o diagnóstico. Ele não isola uma única causa para todo boot lento ou ANR.

PREFLIGHT REUTILIZÁVEL

Cheque a execução antes da feature

O primeiro gate deve abrir uma tela no alvo de teste e executar uma asserção pequena. Escolha a máquina e o dispositivo antes de montar uma suíte completa.

LINUX / NO MESMO CONTEXTO QUE INICIA O AVD

```
"$ANDROID_HOME/emulator/emulator" -accel-check
df -h "$ANDROID_AVD_HOME"
"$ANDROID_HOME/platform-tools/adb" devices -l
tools/android-preflight.sh DEVICE_SERIAL
```

Defina o caminho gravável de dados do AVD antes do comando de espaço. Use um serial explícito e confirme o boot. Compare o API reportado com o da matriz de teste. Se você usa um telefone físico, o passo de aceleração do emulador fica fora da execução.

Ponto de controle	Evidência de saída
Ferramentas	Gradle Wrapper, JDK, SDK, imagem, ABI e espaço reais. Use um cache gravável no contexto do build.
Dispositivo	ADB no estado device, serial esperado e sys.boot_completed=1. Confirme ro.build.version.sdk.
Smoke	Instale o artefato correto, abra uma tela e execute uma asserção. Guarde comando e resultado com a revisão.
Falha do ambiente	Se a ferramenta não inicia, registre o erro e isole host, sandbox, SDK, runner, disco e app antes de alterar código.

Melhoria localizada no script

O preflight atual imprime Java e API. Para usá-lo como gate automatizado, faça a configuração declarar JDK e API esperados, compare os valores e imponha timeout. Mantenha a execução em Bash e use um serial de teste para evitar atuar em outro aparelho.

VERIFICAÇÃO

O que cada teste comprova

Verificação	Resultado, revisão e limite
Primeira entrega	Histórico 2a133da: 6 JVM, teste Compose compilado, primeiro AVD offline. Auditoria seguinte: 1 Compose passou em API 35 e a rotação falhou.
Fechamento local	Histórico 14ab318: 9 JVM, lint debug/release com 0 erros e 7 avisos por variante. XML JVM local retido confirma 9 sem falhas, erros ou skips.
Dispositivo virtual	Histórico: 5 testes em API 26 x86 em 33,588 s e 5 em API 36 x86_64 tablet em 108,407 s, com páginas de 16 KiB no segundo alvo. É a mesma suíte em duas configurações.
Empacotamento	Histórico: Bundletool validou o AAB, 87 APKs passaram no alinhamento ZIP e houve instalação de splits com assinatura descartável.
Pacote local agora	Hashes do APK/AAB coincidem com PROOF. Verifiquei assinatura debug, alinhamento ZIP do APK, metadados 26/36 e ausência de INTERNET no APK. O AAB local está sem assinatura.
Código e specs agora	Comparação TypeScript/Kotlin, perguntas e imagens passou. A CLI 1.16.0 validou 2 specs e 2 changes arquivadas em modo estrito.
CI remoto agora	Run 42590362068, commit 9ce150a, concluiu com sucesso. Inclui build debug/test APK, JVM, lint e AAB. O workflow não executa os testes de dispositivo.

ARTEFATOS LOCAIS / SHA-256

```
APK debug
2a075bec7b6c9903532b125614857fc620c5aa01976605cd82d2cce3b5933750
AAB release sem assinatura
921cf421f1459faba8a3ea1dc5b9c2f32f64bef6cfc2c40f088214846557dae0
```

Lacunas materiais

Ainda faltam evidências próprias de Gradle Sync pela interface do Studio, aparelho físico, TalkBack, atualização por faixa da Play e retenção/transferência entre dispositivos conforme a política prometida.

GITHUB E CI

O gate que faltou no runner

O primeiro pipeline chamou sdkmanager sem preparar as ferramentas Android. A saída 127 ocorreu antes que o Gradle pudesse testar o aplicativo.

Run	O que o histórico e a consulta atual mostram
42520061733	Falhou na etapa de SDK para 14ab318. A causa confirmada foi a ausência de sdkmanager no PATH do runner.
42520816576 / 42520927733	Push e PR do commit 4a26b21 passaram após configurar o SDK explicitamente.
42521377709	A execução pós-merge de 236fa5e, que antes estava em andamento, concluiu com sucesso.
42590362068	O HEAD auditado passou o job Android checks. A etapa de SDK e a de build, JVM, lint e AAB estão verdes.

Fluxo recomendado para as próximas mudanças

Abra uma branch curta a partir da main para cada change com escopo verificável. Prepare PR com contrato, evidência, lacunas e riscos de rollout. Preserve main como ponto de integração e configure checks obrigatórios conforme a política do projeto.

O runner atual configura JDK 21, Gradle e Android SDK antes dos comandos do app. Um gate de emulador ou aparelho pode ser separado e proporcional ao risco. Verifique se os actions e seus pins seguem a política de atualização e cadeia de suprimentos da equipe.

O histórico também registra .git/index.lock e FETCH_HEAD bloqueados pelo sandbox. Teste a escrita no diretório de execução apropriado antes da etapa de Git e trate permissão de workspace, permissão do índice e autorização remota como verificações distintas.

> **CI e a cadeia de confiança**
>
> SDK pronto → wrapper validado → build → teste/lint → artefato identificado → gate de dispositivo quando exigido. Um job que monta o APK de teste confirma compilação. A instrumentação executada precisa de um resultado próprio.

ACHADOS ATUAIS

As lacunas que o CI não cobre

O HEAD de 09/11 trouxe mudanças de documentação e de bootstrap. Elas exigem checks diferentes dos usados para o código Android.

Achado	Impacto, ação e critério de aceite
A1 · README contraditório	O README Android diz que resultados estão no localStorage do navegador e depois descreve DataStore corretamente. Também liga .etc/LICENSE, caminho inexistente no Android. Corrigir a seção e apontar para LICENSE na raiz. Conferir links e texto por plataforma.
A2 · Clone sem setup de skills	As 13 pastas e o lockfile foram removidos do Git. No checkout local vi só cinco entradas OpenSpec em .agents/skills, e nenhum AGENTS.md na raiz. Documentar instalação com origem/revisão e provar descoberta num clone limpo.
A3 · Relatos históricos sem contexto novo	PROOF e auditoria de fechamento ainda dizem que o workflow remoto não rodou. Acrescentar uma atualização datada com o run e o hash; preservar o que se sabia na época.
A4 · Gate manual incompleto	Os emuladores cobrem várias fronteiras, mas o canal Play, o dispositivo físico e o uso com TalkBack precisam de evidência específica. Definir responsáveis e critérios em PRODUCTION.

A1 é um defeito de documentação. A2 é um risco de reprodução do processo em novo ambiente, mesmo quando arquivos ignorados persistem localmente. A3 é atualização da evidência. A4 é um gate de release, com ações do responsável.

> **Como impedir a repetição**
>
> Antes do PR, abra o README como usuário da plataforma, confirme links e comandos num clone novo e procure texto importado que descreve armazenamento, permissões ou distribuição de outro alvo.

SEGURANÇA E PRIVACIDADE

O caminho do resultado

Um resultado pode expor atitudes políticas. A auditoria acompanhou o dado da resposta em memória até o armazenamento e a cópia explícita.

Fronteira	Controle observado e limite
Sessão incompleta	As respostas ficam no ViewModel. Sair do teste ou perder o processo descarta o estado incompleto. A mudança de configuração mantém a sessão em memória.
Resultado salvo	DataStore retém último resultado, data e idioma. O fluxo só anuncia o novo resultado depois do commit e preserva o anterior em falha de escrita.
Backup e migração	O manifesto desativa backup e declara exclusões de nuvem e transferência para a store usada. O comportamento efetivo em dispositivos e fabricantes deve ser exercitado separadamente.
Clipboard	A cópia é uma ação do usuário. O sinal de conteúdo sensível pode suprimir a prévia. O feedback some em dois segundos, mas isso não apaga o clipboard nem controla o app que receber a colagem.
Rede e entrada	O APK debug inspecionado não declara INTERNET. O launcher é exportado por finalidade. O relatório anterior verificou receiver protegido no release. Links externos seguem o toque do usuário.

Próxima melhoria com valor de produto

Especifique uma opção de apagar o último resultado no próprio app. Defina confirmação, falha da operação, efeito no botão Ver Resultado e ausência do dado após reabrir. Se incluir Sharesheet, documente o que será transferido antes do envio.

> **Antes de publicar**
>
> Reconcile termos, privacidade, README e ficha da Play com o APK efetivo. Teste sem usar respostas reais em logs, capturas ou artefatos compartilhados. Verifique a origem e as permissões de distribuição das imagens e do catálogo.

TESTE PESSOAL

Abra no Studio ou instale no celular

Neste Linux, Android Studio está em /opt/android-studio e o SDK em /home/dever/Android/Sdk. Abra a raiz castas-android como projeto Gradle.

Opção	Passos e verificação
Android Studio	Em Settings, escolha o JDK 21 para Gradle. No SDK Manager, confirme plataforma 36, Build-Tools 35.0.0 e Platform-Tools. Faça Gradle Sync, selecione app e um alvo no Device Manager e use Run.
Emulador	Escolha uma imagem API 26 para o mínimo e uma API 36 para o alvo. Use preflight no contexto anfitrião. Confirme o serial, o boot e a API antes de instalar.
Celular USB	Ative opções de desenvolvedor e depuração USB, conecte cabo de dados e aceite a chave RSA no aparelho. Identifique o serial em adb devices -l.
Outra IDE	Use a raiz Gradle e os mesmos comandos em terminal no editor. IntelliJ ou VS Code podem apoiar edição e navegação. Para Compose Preview, Device Manager e profiler, use o suporte Android disponível no Studio.

TERMINAL / RAIZ DO REPOSITÓRIO, JDK 21 CONFIGURADO

```
./gradlew :app:assembleDebug
adb devices -l
adb -s DEVICE_SERIAL install -r \
  app/build/outputs/apk/debug/app-debug.apk
```

Substitua DEVICE_SERIAL pelo alvo escolhido. Há um APK debug local verificado pelo hash de PROOF. Se usar esse arquivo, a etapa de build pode ser dispensada. O AAB é insumo do canal de distribuição.

> **Assinatura e dados de teste**
>
> Uma instalação com o mesmo ID e chave diferente não aceita atualização sobre a existente. Use um aparelho de teste ou preserve o resultado antes de remover a instalação. Desinstalar apaga os dados locais. Use respostas fictícias.

COMPATIBILIDADE E UI

Android 8, tablets e acessibilidade

O APK verificado declara minSdk 26 e targetSdk 36. O mínimo controla a instalação. O target define a faixa de comportamento e exigências de publicação adotadas pelo app.

Cenário	Evidência e próxima checagem
Android 8.0	API 26 foi exercitada em AVD x86. A fonte EmojiCompat empacotada permitiu verificar um emoji recente específico. O catálogo é filtrado para renderização no alvo.
Android 16 e tablet	A suíte rodou em um tablet virtual 1280 × 800 dp com páginas de 16 KiB. O app usa um APK, largura máxima de conteúdo e rolagem.
Janela compacta	Um teste cobriu resultado a 280 dp com fonte a 200%. Para validar a promessa de todas as telas, percorra também teste, seletor e páginas longas com texto ampliado.
Acessibilidade	Os testes observam semântica e ações Compose. Faça uma jornada real com TalkBack, foco após avanço e leitura do progresso. Confira contraste em cada estado e locale pertinente.

Um app para telefone e tablet

Comece com a mesma base e adapte o layout à largura real da janela. Teste rotação, redimensionamento e divisão de tela. Use duas colunas quando ajudarem a tarefa e mantiverem navegação e leitura claras. Uma variante separada exige uma necessidade de produto ou distribuição que sustente seu custo.

Para suportar Android 8 em mudanças futuras, preserve minSdk, inspecione a dependência efetivamente empacotada, use guardas por versão quando necessário, rode lint e repita a jornada no alvo mínimo.

> **Na coleção**
>
> Use ui-design para o contrato adaptativo e android-ui-testing para a instrumentação. Preserve o gate humano de tecnologia assistiva quando a tarefa depender dele.

ENTREGA E PLAY

O caminho até o usuário

O pacote local foi preparado para assinatura. A próxima decisão é validar o mesmo artefato que será entregue pelo canal escolhido.

Passo	Saída verificável
1. Proveniência	Confirme package ID, versionCode, direitos das imagens e avisos aplicáveis. Vincule o código-fonte correspondente e os checks ao release exato.
2. Assinatura	Defina o responsável pela chave de upload e seu backup. Gere o AAB assinado, separe segredos do Git e configure Play App Signing.
3. Requisitos da loja	Prepare descrição, screenshots, classificação, URL pública de privacidade e Data safety com base na variante distribuída. Confira as exigências da conta na Play Console.
4. Faixa de teste	Use primeiro teste interno. Instale e atualize pelo canal em telefone físico. Verifique resultado anterior, idioma, funcionamento offline, cópia e remoção de dados.
5. Rollout e suporte	Registre versão implantada, dono de suporte e sinais de problema. Planeje hotfix com versionCode maior quando reverter o binário não recuperar dados ou instalação.

A política consultada exige API 36 para novas submissões e atualizações de apps de celular desde 31/08/2026. Confirme os requisitos vigentes no dia do envio. Para contas pessoais criadas após 13/11/2023, a regra consultada exige teste fechado com pelo menos 12 participantes inscritos continuamente por 14 dias antes de solicitar acesso à produção.

> **16 KiB e arquitetura**
>
> Alinhamento do pacote, segmentos ELF e instalação num Android com páginas de 16 KiB medem coisas distintas. O fechamento observou os três para o candidato. Repita a inspeção quando bibliotecas nativas, AGP ou empacotamento mudarem.

SKILLS / HISTÓRICO

Instalada, lida e aplicada

O caso teve fases diferentes. Use os registros do momento em que cada ação ocorreu.

Fase	Evidência e conclusão
Início	A CLI criou a raiz OpenSpec. A integração Codex encontrou EROFS. O histórico registra leitura direta das skills Teknolista e de métodos da cópia da web. Isso sustenta consulta, não a ativação nativa de cada skill.
Fechamento	Comparei os 13 SKILL.md rastreados em 14ab318 com 547b47b da coleção. Os 13 coincidem byte a byte. O lockfile do Android identifica só Teknolista.
Métodos usados	Cinco pilares, contrato de UI, Gradle/DataStore, testes de unidade, integração e UI, segurança e revisão Standards/Spec têm efeitos rastreáveis. Koin, WorkManager e quiz não eram necessários.
Origem externa histórica	A cópia de referência web serviu para implementação, diagnóstico, desenho, testes e revisão. O fetch upstream falhou no início. O histórico não prova instalação dessa coleção no Android nem TDD com vermelho e verde por cenário.
HEAD auditado	Os arquivos versionados e o lockfile foram removidos. O diretório local contém cinco skills OpenSpec. Um clone limpo precisa recuperar a coleção e testar descoberta na sessão real.

O que contar como uso

Registre escopo e revisão de instalação, seleção explícita e o resultado específico da skill. Exemplo: Android Integration Testing produz um teste que reabre o DataStore. A presença da pasta ou a lista de skills do host não demonstram esse efeito.

SKILLS / COLEÇÃO ATUAL

O que a coleção já cobre

A edição de 21 skills em 95abba6 absorveu métodos úteis em workflows próprios. O novo projeto pode partir só da coleção Teknolista, além da integração escolhida do OpenSpec.

Capacidade	Dono e evidência de aplicação
Produto e domínio	penta-pro. Registra fontes, divergências, decisão, cinco pilares e verificação com OpenSpec como dono da change.
Investigação e falha	technical-research e diagnose-defect. Separam hipótese e prova, com uma reprodução, controle e menor teste que diferencia causas.
Execução e testes	software-delivery, behavior-testing e os helpers Android pertinentes. Entregam uma fatia verificável e tratam falhas no limite real.
Interface e segurança	ui-design e cybersec. Derivam testes de estados, entradas, privacidade e efeitos da fronteira.
Integrações	http-api e mcp-server. Use conforme o transporte e a autorização que o serviço exige. O Castas local não precisava dessas superfícies.
Revisão e publicação	peer-review e git-delivery. Revisão dos padrões e do contrato, snapshot, checks e publicação autorizada. pr-quiz serve como exercício separado quando solicitado.
Manutenção	agent-instructions. Corrige instrução, descoberta ou gate com uma regressão e um critério observável.

As oito skills acrescentadas após o Castas são software-delivery, technical-research, diagnose-defect, behavior-testing, http-api, mcp-server, git-delivery e agent-instructions. O histórico do Castas explica a motivação. A eficácia dessas versões no app exige um ensaio separado.

> **Escolha de manutenção**
>
> Priorize referências de preflight, paridade e release dentro dos donos existentes. Um novo entrypoint precisa de uma tarefa distinta, recorrente e um resultado que ainda não tenha dono.

SDD, OPENSPEC E TUTORIAL

O contrato que chegou à entrega

O projeto adotou SDD na prática ao definir o delta antes do código e revisar o resultado contra esse contrato. O percurso da integração do agente foi parcial no início.

Etapa do guia	O que o Castas demonstrou
Instalação e ativação	OpenSpec CLI inicial funcionou e a escrita da integração falhou. As skills foram consultadas diretamente; a instalação da coleção própria foi confirmada no fechamento.
Proposta, delta e design	Contrato no OpenSpec, cinco pilares em docs e UI em documento cumulativo. A discrepância Likert mostrou por que citar a fonte executável é necessário.
Apply e verificação	A primeira entrega ficou em 6/12 tarefas. O fechamento concluiu 12 de comportamento e 4 de release. As fronteiras de teste foram separadas antes de arquivar.
Archive e continuidade	Dois arquivos de change e duas specs consolidadas passaram na validação estrita. O HEAD rastreia openspec/ e a regra atual de ignore não cobre novos artefatos nesse caminho.

TERMINAL / CONFERÊNCIA DO ESTADO DO PROJETO

```
openspec list
openspec list --specs
openspec validate --all --strict --no-interactive
openspec validate --archived --strict --no-interactive
```

O guia atualizado já explica descoberta, seleção explícita e aplicação observável. Acrescente ao próprio processo uma ficha de portabilidade com fonte, divergência, decisão, teste e prova, mais um preflight real antes do apply. Separe aceite do contrato, archive, pacote e release.

> **Regra de versionamento**
>
> Versione specs, changes e configuração do OpenSpec. Se a instalação de skills é local, fixe a versão e documente como recriá-la. Antes do commit, use git check-ignore nos novos caminhos que deveriam entrar no repositório.

MODELOS E TAMANHO DO TRABALHO

O que o caso diz sobre Luna

O histórico registra GPT-6 Luna com esforço low na primeira implementação e GPT-6.1 Sol com esforço high na auditoria e no fechamento. Outros turnos posteriores voltaram a usar Luna.

Os modelos trabalharam em tarefas e ambientes diferentes. O fechamento já tinha código, achados reproduzidos, SDK preparado e uma definição mais precisa do aceite. Esse histórico não permite atribuir a diferença de qualidade a uma troca de modelo. O sandbox e o runner impunham limites reais.

Papel recomendado	Como encaixar no próximo projeto
GPT-6.1 Sol	Modelo principal de equilíbrio para coordenação, contratos e implementação recorrente. Preserve checks explícitos e checkpoints.
GPT-6 Astra	Use quando a ambiguidade, arquitetura, segurança ou revisão crítica justificar mais capacidade. Forneça contrato, diff e evidências.
GPT-6 Luna	Use para fatias pequenas e reproduzíveis, como converter catálogos, portar vetores, testar uma fronteira ou corrigir um erro com reprodução pronta.

Um pedido amplo, várias entregas verificadas

Um prompt pode autorizar o app completo. Execute em fatias que cruzam a interface e o domínio relevante. No Castas, comece por uma tela em aparelho; depois um fluxo curto com persistência; depois paridade, recuperação e release. Use checkpoints e contexto fresco para revisar.

> **Como medir o efeito de um modelo**
>
> Fixe revisão, ferramentas, tempo, critérios e artefatos iniciais. Repita as mesmas tarefas por modelo e esforço. Meça cenários aprovados, defeitos confirmados, falsos positivos, retrabalho, cobertura de runtime, custo e tempo. Escolha o modelo pela tarefa e pela evidência.

FÁBRICA DE SOFTWARE / FLUXO

Sete gates para repetir o processo

Use a coleção como instruções selecionáveis. A execução depende de ferramentas, permissões e evidência próprias do projeto.

Gate	O que autoriza avançar
0. Preparar	Clone limpo, raiz e versão fixadas. Inventário de skills, integração OpenSpec, comandos e preflight do alvo. Uma tela ou operação real executa.
1. Especificar	PentaPRO registra objetivo, exclusões, riscos e decisões. OpenSpec possui o delta e tarefas. Contradições têm dono e critério de resolução.
2. Dividir	Software Delivery seleciona fatia com início, efeito e verificação. Fixe próxima tarefa, dependências e saídas esperadas.
3. Construir	Selecione as skills das fronteiras. Rode o teste mais próximo do efeito. Compare com o baseline e preserve resultados que já existem.
4. Revisar	Peer Review verifica requisitos e padrões. Cybersecurity entra nos riscos materiais. Reavalie as correções no snapshot final.
5. Consolidar	Evidência vinculada à revisão, critérios satisfeitos, tarefas reconciliadas e archive validado. A spec consolidada representa a mudança aceita. Uma change arquivada continua vinculada ao escopo testado.
6. Entregar	Git Delivery verifica conteúdo, destino e hash. O responsável aprova assinatura, publicação e rollout. Observe instalação e recuperação no canal.

Interrupção saudável

Se faltar uma skill exigida, toolchain, decisão, aprovação ou teste de runtime, mantenha o gate pendente. Continue o trabalho independente que preserva o contrato. Registre o que precisa mudar para retomar.

FÁBRICA DE SOFTWARE / APLICAÇÃO

Um contrato de execução curto

Em cada turno, selecione as skills exigidas pelo host e identifique a change. Use um pedido específico com artefatos, verificação e condição de parada.

CHAT / EXEMPLO PARA A PRÓXIMA FATIA

```
$openspec-apply-change
$software-delivery
Apply task 1.2 of the selected change only.
Use the accepted delta and the actual project toolchain.
Name the required specialist skill and select it when needed.
Run the relevant check. Record command, revision, target,
result and remaining gaps in PROOF and the change record.
Pause the dependent task if a material gate is unavailable.
```

Se o host aceitar uma menção por vez, divida preparação e execução sem mudar o ID da tarefa. Confira disponibilidade pelo seletor, versão pelo SKILL.md, e aplicação pela saída observável. Uma frase do agente sobre ativação vale menos que um registro do host e um check que realmente exerceu a fronteira.

BOOTSTRAP / RAIZ DE UM PROJETO NOVO

```
openspec init . --tools codex --profile core --language English
npx skills@latest add teknolista/skills -a codex
```

Se o projeto já tem integração, confira a versão e o perfil antes de alterar. Para fixar a edição, copie os diretórios completos das skills de uma revisão conhecida. Consulte o guia SDD atualizado para seleção e descoberta no Codex.

Ferramentas por necessidade

No Castas, o núcleo é Studio, SDK/ADB, Gradle Wrapper, OpenSpec e Bundletool. Escolha um emulador acelerado, aparelho de teste ou runner adequado quando precisar de execução real. Para web, Playwright pode complementar teste manual de navegação e acessibilidade. Para iOS, use Xcode em macOS.

> **Onde colocar uma melhoria**
>
> Projeto: comandos, matriz, política e segredos por nome. Skill: procedimento recorrente e decisão observável. CI: enforcement de um check que pode falhar sozinho. Guia: roteiro e exemplos para a pessoa. Evite manter a mesma regra em quatro lugares.

OUTROS ALVOS

O que reaproveitar fora do Android

Reutilize o contrato de domínio, catálogos, vetores de cálculo e critérios de aceite. Teste instalação, persistência, entrada e recuperação na plataforma que o usuário realmente usa.

Alvo	Primeira entrega, ferramentas e skill
iPhone e iPad	Swift e SwiftUI nativos se o objetivo for uma experiência iOS nativa. Rode Xcode/Simulator em macOS, XCTest/XCUITest e um iPhone. Valide Dynamic Type, VoiceOver, persistência e compartilhamento. Distribua beta por TestFlight conforme a conta.
Web em navegadores	O Castas Web já fornece React/TypeScript como referência. Teste rotas, teclado, foco, zoom, idioma, armazenamento por origem e perda de rede conforme o contrato. Use ui-design e behavior-testing, com matriz de navegadores suportados.
Desktop local	Escolha pela integração. UI nativa por SO atende necessidades específicas. Uma base comum pode usar Qt ou uma UI web empacotada quando a plataforma e o requisito permitirem. Teste instalador, atualização, atalhos, armazenamento e acessibilidade em macOS, Windows e Linux.
Serviços e ferramentas	Escolha http-api ou mcp-server só se o produto pedir serviço. Exercite identidade, efeitos reais, repetição, timeout e autorização. Use behavior-testing para a fronteira e cybersec nos riscos materiais.

PentaPRO, OpenSpec, Software Delivery, Diagnose Defect e Peer Review mantêm os papéis. As referências condicionais da plataforma cobrem o que muda. Considere um helper iOS ou desktop após um piloto comprovar uma lacuna recorrente e distinta.

> **Primeiro teste em cada alvo**
>
> Empacote uma tela ou operação, instale no ambiente alvo e verifique um cenário antes de expandir o escopo. Para três sistemas desktop, três builds e instalações dão evidência melhor que uma suposição de portabilidade.

PLANO DE AÇÃO

O que fazer primeiro

Ordem	Ação, responsável sugerido e critério de conclusão
Agora · documentação	Mantenedor corrige README Android e acrescenta CI datado em PROOF. Nenhum link quebrado; descrição de armazenamento e privacidade coincide com o app e o pacote.
Agora · bootstrap	Responsável pelo fluxo escolhe revisão da coleção e documenta instalação. Clone limpo descobre OpenSpec e skills relevantes. Um pedido de leitura identifica a próxima tarefa e o artefato correto.
Antes do release	Responsável pelo pacote resolve direitos, assinatura e política da loja. Um usuário de teste instala e atualiza pela faixa escolhida em aparelho físico. O release fica ligado ao hash e à evidência.
Próxima change do Castas	Escolha um problema de usuário, como apagar o resultado no app. Defina o estado anterior, erro, efeito e reabertura. Implemente e valide em uma change curta.
Próximo projeto	Execute o preflight antes da primeira fatia. Reuse a matriz de paridade e os gates de evidência. Faça um teste deliberadamente falhar para verificar o oráculo quando usar TDD.
Após dois ou mais casos	Compare resultados com a coleção atual e ajuste somente as instruções que mudam o resultado. Use o procedimento de avaliação para testar modelo, host, custo e regressões.

Checklist antes de declarar a etapa concluída

O dono do contrato está claro. A skill exigida foi selecionada e seu efeito aparece no artefato. O comando rodou no alvo correto. A revisão e o resultado estão registrados. Os limites e o próximo gate têm responsáveis.

> **A lição que liga todos os achados**
>
> A melhor prova aparece quando o agente executa cedo o caminho que o usuário vai percorrer e liga esse resultado ao requisito. Foi esse encadeamento que revelou rotação, contraste e setup, e que permite repetir a entrega com menos retrabalho.

APÊNDICE / REFERÊNCIAS

Como reencontrar a evidência

Repositórios, auditorias e CI

Castas Android / snapshot auditado 9ce150a

Primeira auditoria / estado da entrega inicial

Auditoria de fechamento e matriz de verificação

PROOF / comandos, hashes e cobertura

Castas Web / correção do wireframe Likert

GitHub Actions / HEAD 9ce150a

Coleção Teknolista / versão 95abba6

Documentação oficial consultada

Android Emulator / aceleração e KVM

Android / páginas de 16 KiB e distribuição

Android / assinatura e Play App Signing

Play Console / target API e datas de submissão

Play Console / teste fechado para contas pessoais

Android / telas adaptativas e dimensões de janela

OpenSpec / versionamento em equipes

Codex / seleção e descoberta de skills

OpenAI / modelos e perfis de uso

Apple / requisitos do Xcode

Apple / TestFlight

Android / proteção de conteúdo no clipboard

Playwright / matriz de navegadores

Material local

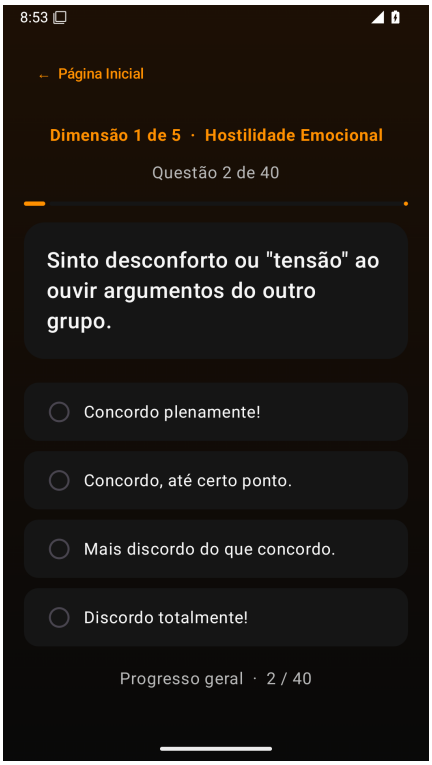
Conversa de desenvolvimento 01a1b8ae-4a91-7881-93b0-658937952e86, os dois prompts de auditoria de 08/11 e o guia SDD com OpenSpec atualizado em 09/11. O histórico ancora as etapas antigas. Os links acima apontam para as evidências de produto e execução que podem ser revisitadas.

APÊNDICE / EVIDÊNCIA VISUAL

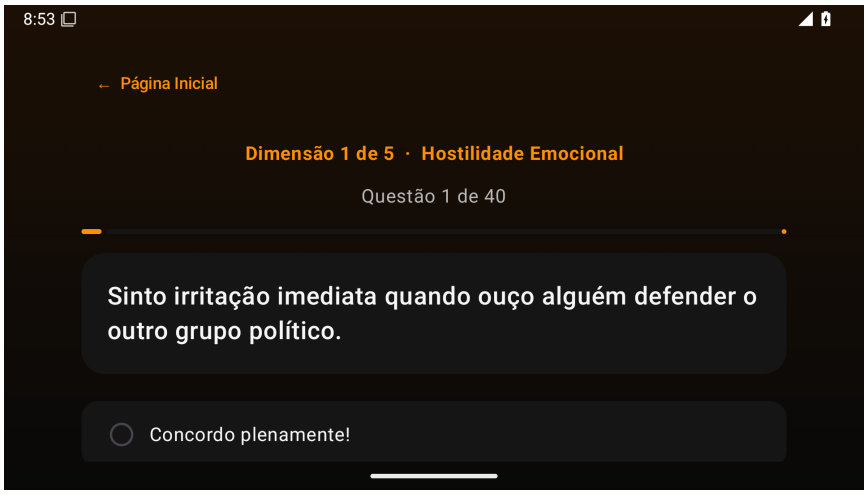
A rotação que expôs o defeito

A primeira auditoria reproduziu a perda de estado em um AVD API 35 com dados de teste. As duas capturas abaixo são daquele ensaio, antes da correção.

ANTES DE GIRAR / QUESTÃO 2



APÓS GIRAR / QUESTÃO 1



O Android recriou a Activity. O estado guardado na composição reiniciou a sessão. O contador e o enunciado da captura confirmam a mudança.

Como a correção foi verificada

O fechamento moveu a sessão para o ViewModel e registrou um check separado que girou o dispositivo mantendo a pergunta 2. A spec distingue essa recriação da morte do processo e da saída voluntária do teste.

Fonte das capturas: primeira auditoria em 08/11, Android 2a133da, AVD API 35. A evidência de correção está no relatório de fechamento e na matriz PROOF.